

Fehlerbehandlung mit Optional und Result

Carsten Gips (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Optional<T> - “Vielleicht gibt es einen Wert”

```
public class LSF {  
    private Set<Studi> sl;  
  
    public Studi getBestStudi() {  
        if (sl == null) return null; // Fehler: Es gibt noch keine Sammlung  
        Studi best = null;  
        for (Studi s : sl) {  
            if (best == null) best = s;  
            if (best.credits() < s.credits()) best = s;  
        }  
        return best;  
    }  
}  
  
public static void main(String... args) {  
    LSF lsf = new LSF(); Studi best = lsf.getBestStudi();  
    if (best != null) {  
        String name = best.name();  
        if (name != null) { /* mach was mit dem Namen ... */ }  
    }  
}
```

Erzeugen von *Optional*-Objekten

Konstruktor ist `private` ...

- “Kein Wert”: `Optional.empty()`
- Verpacken eines non-`null` Elements: `Optional.of()`
(`NullPointerException` wenn Argument `null`!)
- Verpacken eines “unsicheren”/beliebigen Elements: `Optional.ofNullable()`
 - Liefert verpacktes Element, oder
 - `Optional.empty()`, falls Element `null` war

Important

`null` kann nicht in `Optional<T>` verpackt werden!

LSF liefert jetzt *Optional* zurück

```
public class LSF {  
    private Set<Studi> sl;  
  
    public Optional<Studi> getBestStudi() throws IllegalStateException {  
        // Fehler: Es gibt noch keine Sammlung  
        if (sl == null) throw new IllegalStateException("There ain't any collection");  
  
        Studi best = null;  
        for (Studi s : sl) {  
            if (best == null) best = s;  
            if (best.credits() < s.credits()) best = s;  
        }  
  
        // Entweder Optional.empty() (wenn best==null) oder Optional.of(best) sonst  
        return Optional.ofNullable(best);  
    }  
}
```

Zugriff auf *Optional*-Objekte

```
Studi best, anne;
```

```
// Testen und dann verwenden
```

```
if (!lsf.getBestStudi().isEmpty()) {  
    best = lsf.getBestStudi().get();  
    // mach was mit dem Studi ...  
}
```

```
// Arbeite mit Consumer
```

```
lsf.getBestStudi().ifPresent(studi -> {  
    // mach was mit dem Studi ...  
});
```

```
// Studi oder Alternative (wenn Optional.empty())
```

```
best = lsf.getBestStudi().orElse(anne);
```

```
// Studi oder NoSuchElementException (wenn Optional.empty())
```

```
best = lsf.getBestStudi().orElseThrow();
```

Einsatz mit Stream-API

```
public class LSF {  
    ...  
    public Optional<Studi> getBestStudi() throws IllegalStateException {  
        if (s1 == null) throw new IllegalStateException("There ain't any collection");  
        return s1.stream()  
            .sorted((s1, s2) -> s2.credits() - s1.credits())  
            .findFirst(); // Optional<Studi>  
    }  
}  
  
public static void main(String... args) {  
    ...  
    String name = lsf.getBestStudi() // Optional<Studi>  
        .map(Studi::name) // Optional<String>  
        .orElseThrow();  
}
```

Regeln für *Optional*

1. Nutze `Optional` nur als Rückgabe für "kein Wert vorhanden"
2. Nutze nie `null` für eine `Optional`-Variable oder einen `Optional`-Rückgabewert
3. Nutze `Optional.ofNullable()` zum Erzeugen eines `Optional`
4. Erzeuge keine `Optional` als Ersatz für die Prüfung auf `null`
5. Nutze `Optional` nicht in Attributen, Methoden-Parametern und Sammlungen
6. Vermeiden Sie Muster wie `if (opt.isPresent()) { T v = opt.get(); ... }`.

Important

`Optional<T>` für (normale) Abwesenheit eines Wertes, nicht für Fehler mit Erklärung!

Was ist das Problem mit Exceptions?

```
public static int parsePort(String input) {  
    int port = Integer.parseInt(input);  
    if (port < 1024 || port > 65535) {  
        throw new IllegalArgumentException("Port out of range");  
    }  
    return port;  
}
```

Gibt es eine Möglichkeit, Fehler so zu modellieren, dass sie im Typ sichtbar sind?

Result<E,R>: Fehler als Datentyp modellieren

```
public sealed interface Result<E, R> permits Result.Ok, Result.Err {  
  
    record Ok<E, R>(R value) implements Result<E, R> {}  
    record Err<E, R>(E error) implements Result<E, R> {}  
  
    static <E, R> Result<E, R> ok(R value) { return new Ok<>(value); }  
    static <E, R> Result<E, R> err(E error) { return new Err<>(error); }  
}
```

Umbau unseres Beispiels auf `Result<E,R>`

```
public enum PortError {  
    NOT_A_NUMBER,  
    OUT_OF_RANGE  
}
```

```
public static Result<PortError, Integer> parsePort(String input) {  
    try {  
        int port = Integer.parseInt(input);  
        if (port < 1024 || port > 65535) {  
            return Result.err(PortError.OUT_OF_RANGE);  
        }  
        return Result.ok(port);  
    } catch (NumberFormatException e) {  
        return Result.err(PortError.NOT_A_NUMBER);  
    }  
}
```

Handling auf Aufrufer-Seite

```
public static void foo(String input) {  
    Result<PortError, Integer> result = PortParser.parsePort(input);  
  
    switch (result) {  
        case Result.Ok<PortError, Integer> ok -> {  
            int port = ok.value();  
            IO.println("Starte Server auf Port " + port);  
        }  
        case Result.Err<PortError, Integer> err -> handleError(err.error());  
    }  
}  
  
private static void handleError(PortError err) {  
    switch (err) {  
        case NOT_A_NUMBER -> System.err.println("Eingabe ist keine Zahl.");  
        case OUT_OF_RANGE -> System.err.println("Port muss zwischen 1024 und 65535 liegen.");  
    }  
}
```

- Wann sind Exceptions sinnvoll?

- Unerwartete, außergewöhnliche oder technische Fehler
- “Defensive” Fehlerfälle: I/O-Fehler, Netzwerk weg, Datenbank down

“Ich habe nicht erwartet, dass das passiert.”

- Wann Result/Optional?

- **Optional** für “Wert fehlt, aber es ist kein Fehler”
- **Result** für erwartbare, domänenspezifische Fehlerfälle mit Bedeutung
- Validierungsfehler, Suchergebnisse nicht gefunden, ungültige User-Eingaben

“Das gehört zum normalen Verhalten meiner Funktion.”

1. `Optional<T>`:

- Vermeidet `null` und `NullPointerException`
- Macht normale “kein Wert”-Fälle im Typ sichtbar
- Nicht als universeller Ersatz für jedes `null`, sondern vor allem als Rückgabewert
- Bietet mit `map`, `flatMap`, `orElse*`, `ifPresent` bequeme Werkzeuge, in die Stream-API integriert

2. `Result<E, R>` (oder ähnliche Typen):

- Modelliert Erfolg **oder** Fehler explizit
- Typ zeigt: Funktion kann ein Ergebnis oder einen Fehler liefern
- Vereinfacht Testing: Kein `try/catch` nötig, Fehler sind nur Daten
- Mit `map` und `flatMap` lassen sich Pipelines elegant schreiben

3. Designentscheidungen: Fragen Sie sich: Ist dies ein normaler Fehlerfall?

- Ja → `Optional/Result`
- Nein → `Exception`



Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

***Last modified:** d1bbf0e 2026-06-26 optional: rework screencasts*